

Project Final Grading Rubric

Note to TAS: grade each item from 1 to 4 for each of the four items below. Fractions are fine. Enter into bright space (or save as a Spreadsheet and send to me).

1. Graduate attribute 7. Can the student follow instructions?

	1	2	3	4
5%	Something submitted.	Instructions not followed, but source and report present.	Source code and report present. Source submitted using usual instructions. No test code.	Yes, completely. 1. A reports (PDF) 2. Test cases. 3. Source code using submission guidelines.

2. Report Organization.

	1	2	3	4
35%	Something submitted.	1. A breakdown of responsibilities for all team members. 2. Clear and concise UML diagrams. 3. Detailed instructions for set up. <u>OR</u> Poorly written overall (typos everywhere, poor grammar).	1. A breakdown of responsibilities for all team members. 2. Clear and concise UML diagrams. 3. Detailed instructions for set up. 4. Measurement results for determining zone and drone movement times.	A superb report which includes 1. Table of contents. 2. A breakdown of responsibilities for all team members. 3. Clear and concise UML diagrams. 4. Detailed instructions for set up running and testing. 5. Measurement results for determining zone and drone movement times. 6. Conclusions: Reflection on the project (missing – deduct 0.5). 7. Grammar and spelling are excellent.

3. Source code:

Project Final Grading Rubric

	1	2	3	4
30%	Code is somewhat readable.	Code is readable, but comments are lacking or the naming is atrocious (does not match class diagrams). Add 0.5 if test code included.	1. Properly indented 2. Well commented, 3. Meaningful names, 4. Names match class diagrams. No test case code included or no commenting.	1. Properly indented 2. Well commented (i.e., Javadoc) 3. Meaningful names, 4. Names match class diagrams. 5. Test cases present.

4. Testing

	1	2	3	4
30%	Testing involves running the final submission only. The code “runs” (or NO source code supplied for tests).	Unit and integration testing showing the proper operation of multiple drones on separate machines.	Unit and Integration tests present. The tests show all drones are running. However, tests for faults are lacking or the system crashes when a fault is injected (or the test cases are NOT documented).	Exhaustive unit tests written using a framework such as Junit. Integration testing showing all of the components function together. Acceptance testing showing proper system functioning in the presence of faults. Test cases are documented.